



Bahir Dar University

Department Of Software

Engineering

Course Title : Operating System

Name : Biruh Tesfa

ID : BDU1701728

Section : A

Submitted To : Mr.Wendmu

Submission Date : 20/08/2018 E.C

TABLE OF CONTENTS

1, Introduction	1
2, Objectives	1
3, Requirements	2
3.1, Hardware	2
3.2, Software	3
4, Installation Steps	3
5, Execution of Kernel-Level System Calls:	6
5.1, User Identity and Account Creation	6
5.2, Troubleshooting Connectivity	7
5.3, Restoring the Network and DHCP	8
5.4, Capturing the execve System Call	9
6, Issues (Problems Faced)	10
6.1, Repository Access Failure (Failure Of VM To Access Internet)....	10
6.2, Small Cursor Of VM	11
7, Solution	11
7.1, Network Bridge And Kernel Tracing.....	11
7.2, Solution For 6.2.....	12
8 File system Support	12
9, Advantage and Disadvantage	13

9.1, Advantage.....	13
9.2, Disadvantage.....’’’’’’’’’’	13
10, Conclusion	13
11, Future Outlook / Recommendation	14
12, Virtualization in Modern Operating Systems	14
13, References	16

◆ OPERATING SYSTEM INSTALLATION AND SYSTEM CALL

✧ **Assigned OS: Tiny Core Linux (CorePlus)**

✧ **Environment: Oracle VM VirtualBox (Type-2 Hypervisor)**

1. INTRODUCTION

- In this project, I explored the deployment of a smallest operating system within a virtualized environment. Tiny Core Linux was selected specifically to demonstrate how an OS kernel can function with extreme efficiency by running entirely within a RAM-based file system. This report details the full life cycle of the project—from initial hardware allocation to the sophisticated task of tracing kernel system calls.
- The primary objective of this project was to explore the deployment and operational life cycle of a minimalist Linux distribution within a sophisticated virtualized environment. Tiny Core Linux, specifically the CorePlus variant, was chosen as the subject for this study due to its unique architectural philosophy. Unlike conventional operating systems such as Windows or Ubuntu, which rely heavily on persistent disk-based execution, Tiny Core is engineered to run entirely within a RAM-based filesystem.
- This "Copy to RAM" architecture offers a fundamental demonstration of Operating System efficiency. By loading the entire kernel and necessary extensions into the system's memory, the OS eliminates the latency bottlenecks typically associated with physical or virtual disk I/O. This results in near-instantaneous process execution and a highly responsive user interface, even when allocated minimal hardware resources.
- Throughout this report, I will detail the comprehensive lifecycle of the project. This starts with the strategic allocation of virtualized hardware, proceeds through the nuanced steps of a "Frugal" installation, and culminates in a deep-dive analysis of network restoration and filesystem integrity. By documenting the challenges faced—such as repository isolation—this report serves as a technical guide for deploying high-efficiency systems in a sandboxed, virtualized environment.

2. OBJECTIVES

- **Deployment:** Perform a complete installation of a Linux distribution.
- **Troubleshooting:** Identify and resolve hardware to software communication barriers within a hypervisor.

- **Kernel Analysis:** Demonstrate and analyze the functionality of the `execve()` system call.
- **Verification:** Evaluate the utility of the ext4 journaling filesystem in a virtual environment.

3. REQUIREMENTS

3.1 Hardware Allocation Strategy

For this deployment, accurate resource allocation was paramount to balance guest performance with host stability. The virtual hardware profile was configured as follows:

- **Base Memory (RAM):** 512MB. While modern OSs require 4GB to 8GB, this project proves that a Linux kernel can maintain a full graphical user interface (GUI) on a fraction of that amount.
- **Processors:** 1 CPU core.
- **Virtual Hard Disk:** 2GB. This disk was utilized primarily for the "Frugal" installation files rather than a standard bloated OS installation.

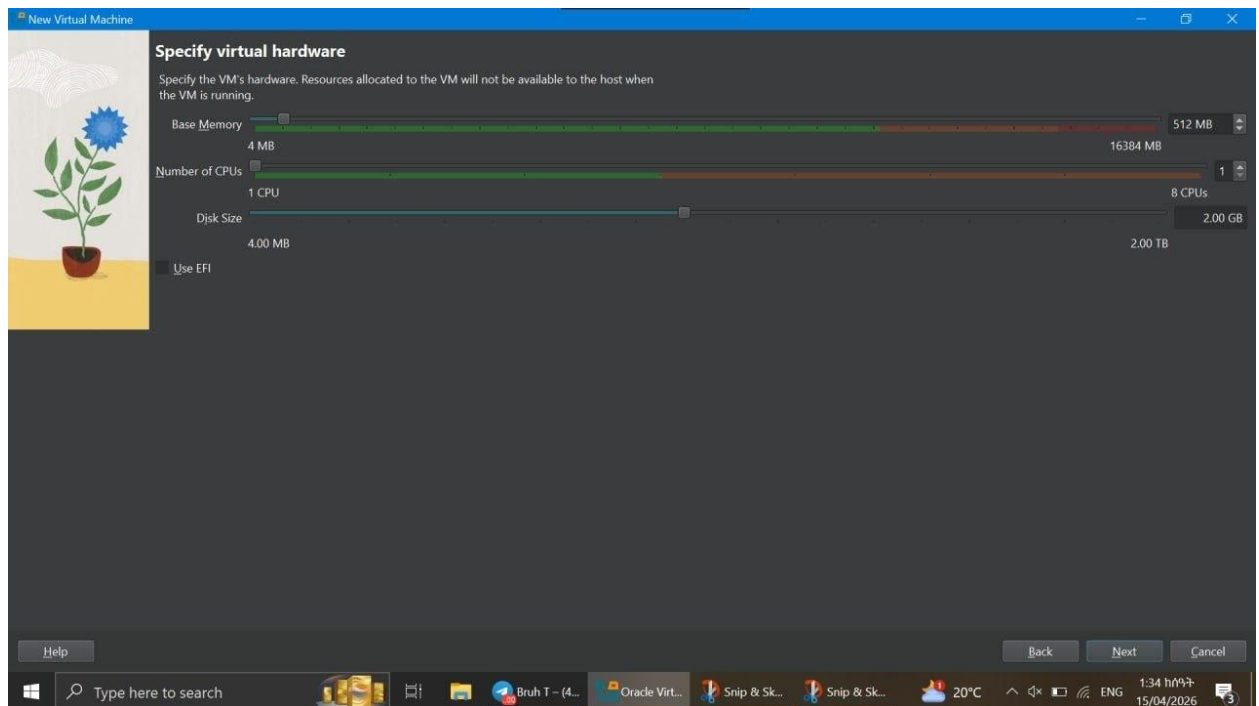


Figure 1: Allocation Of Hardware Devices For The System

- This image displays the virtual hardware profile. I configured the system with 512MB of RAM and a 2GB Virtual Hard Disk, proving that modern Linux kernels can operate on minimal resources.

3.2 Software :

- ✓ Basically we use Tiny Core Linux for our project

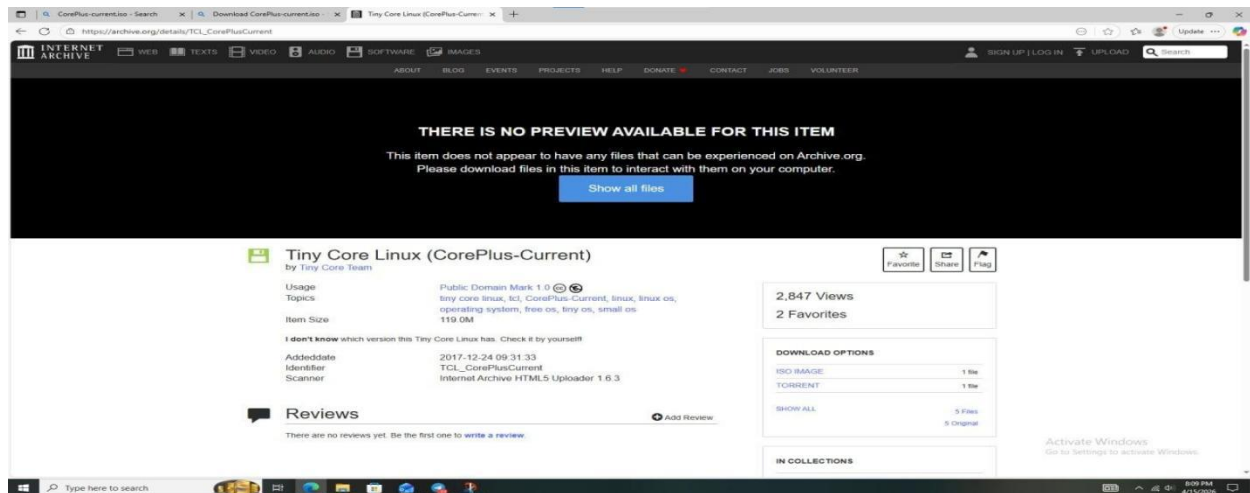


Figure 2: The Downloading page of the Tiny Core ISO.

- ✓ Platform(Environment): Oracle VM VirtualBox was utilized as the Type-2 Hypervisor for this project. A Type-2 hypervisor runs as an application on top of a host operating system (such as Windows 10/11) and provides a hardware abstraction layer for the guest OS.
- ✓ Abstraction Layer: VirtualBox creates virtual versions of CPU, RAM, and Disk, allowing the Tiny Core Linux kernel to believe it is running on real hardware.
- ✓ Resource Isolation: This setup ensures that if the Guest OS crashes or faces a kernel panic during installation, the Host OS remains unaffected.
- ✓ Efficiency: By maximizing hardware use, I was able to run a secondary, fully functional environment without the need for additional physical machines.

4. INSTALLATION STEPS

1. VM Creation: The Virtual Machine was initialized under the name "Biruh_Tesfa_TinyCore" to ensure proper user identification

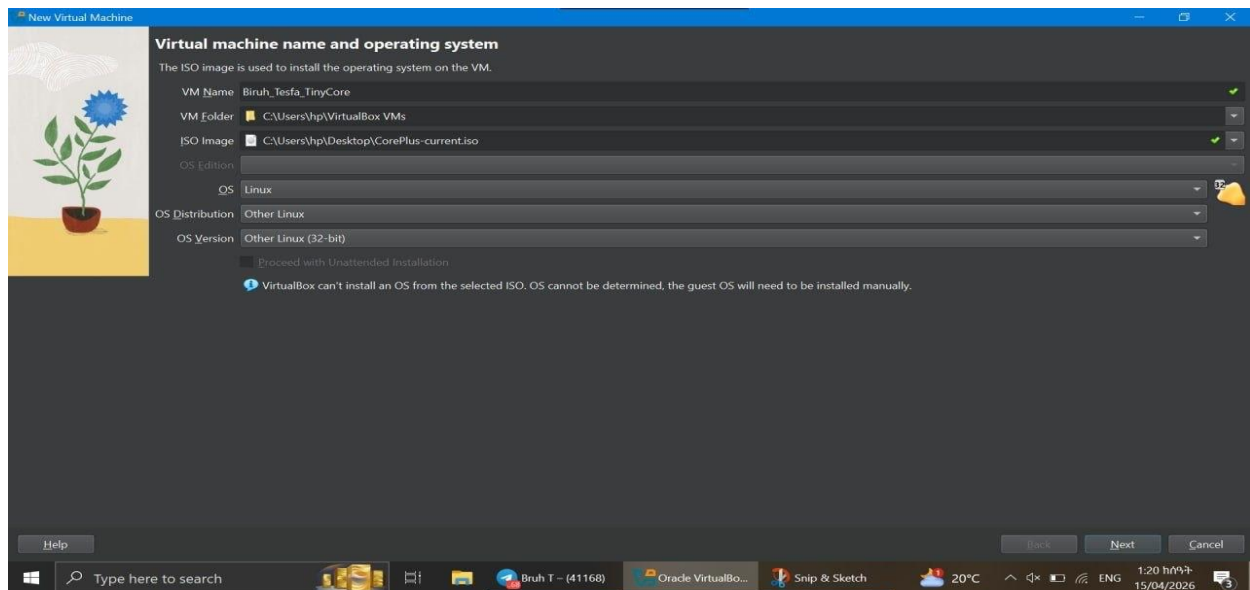


Figure 3: Account Creation

2. Initial Boot: The system was successfully booted into the CorePlus graphical installer environment .



eparationFigure 4: Booting Process

3. Disk Pr: I selected the virtual disk sda for a clean, "Whole Disk" installation.

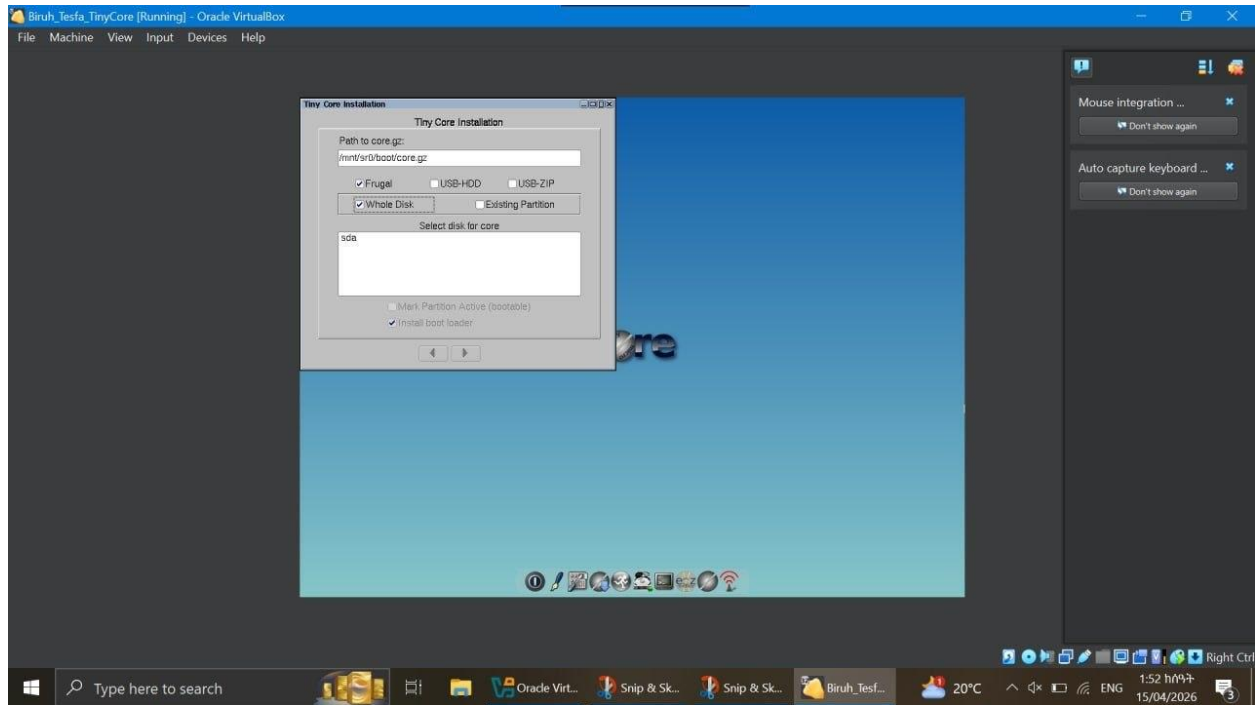


Figure 5: Installation Configuration

4. Filesystem Selection: I opted for the ext4 journaling filesystem to ensure data integrity.

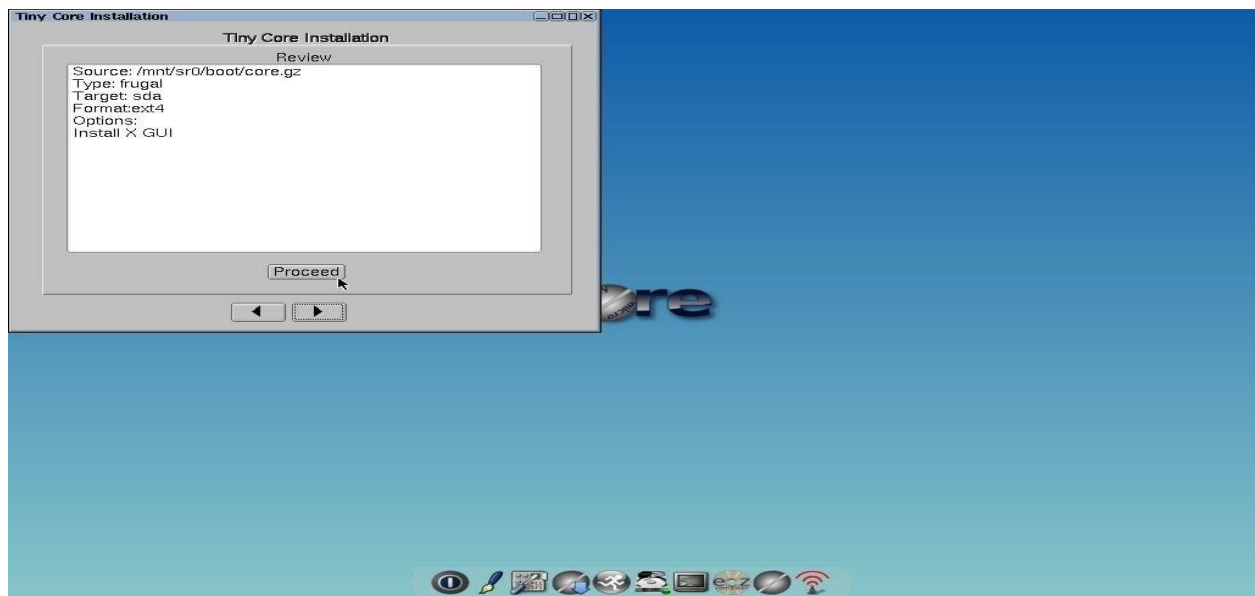


Figure 6 : Shows a review for our installation process and it contain type ,target as well as format (ext 4)

5. Installation Confirmation: The installer finalized the deployment of the bootloader and core system files.

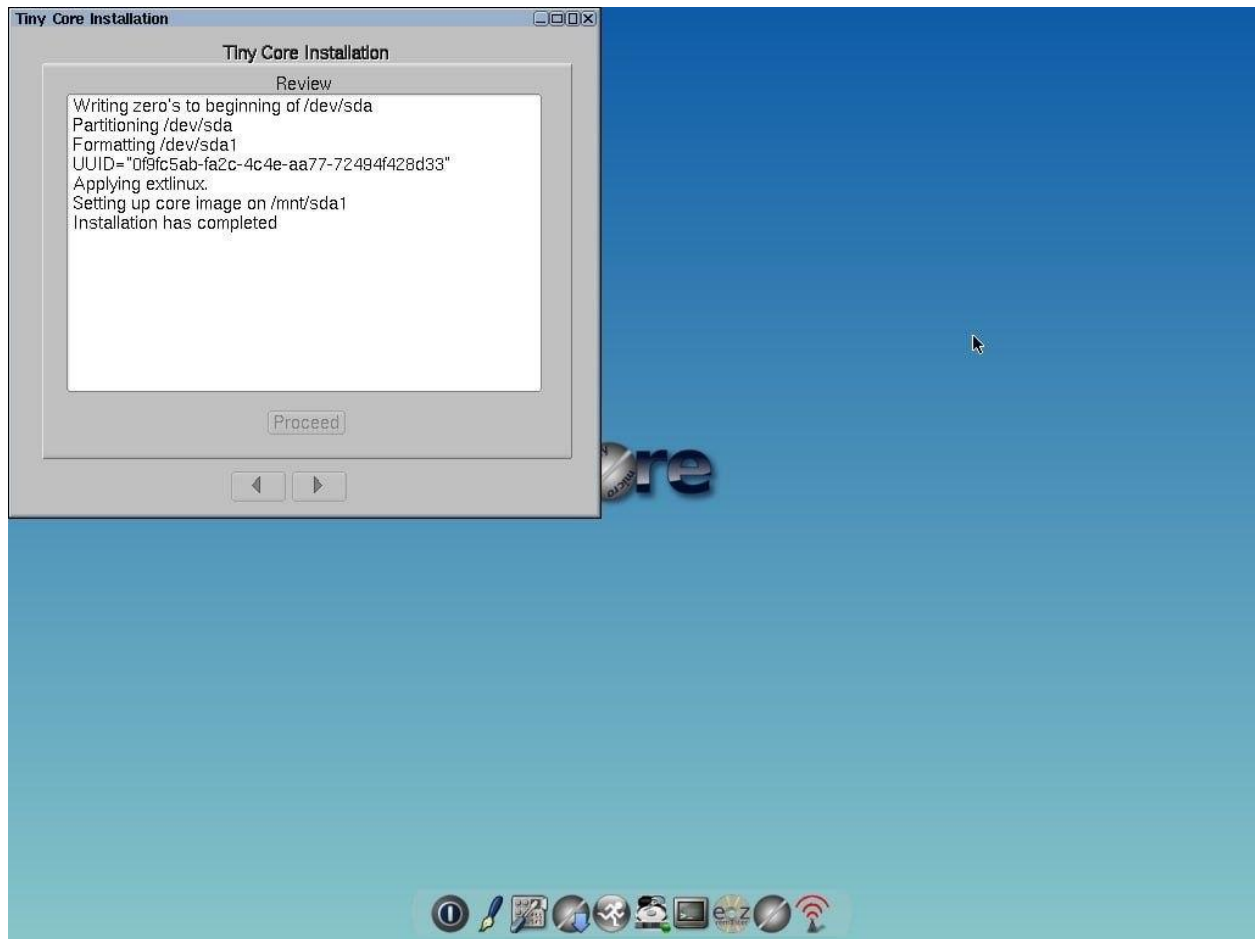


Figure 7: The Installation Completed

5. EXECUTION OF KERNEL-LEVEL SYSTEM CALLS: THE EXECVE METHOD

- After the base installation, I shifted focus to the kernel interface. This stage required resolving network isolation issues encountered during the initial setup to download diagnostic tools.

5.1 Step 1: User Identity and Account Creation

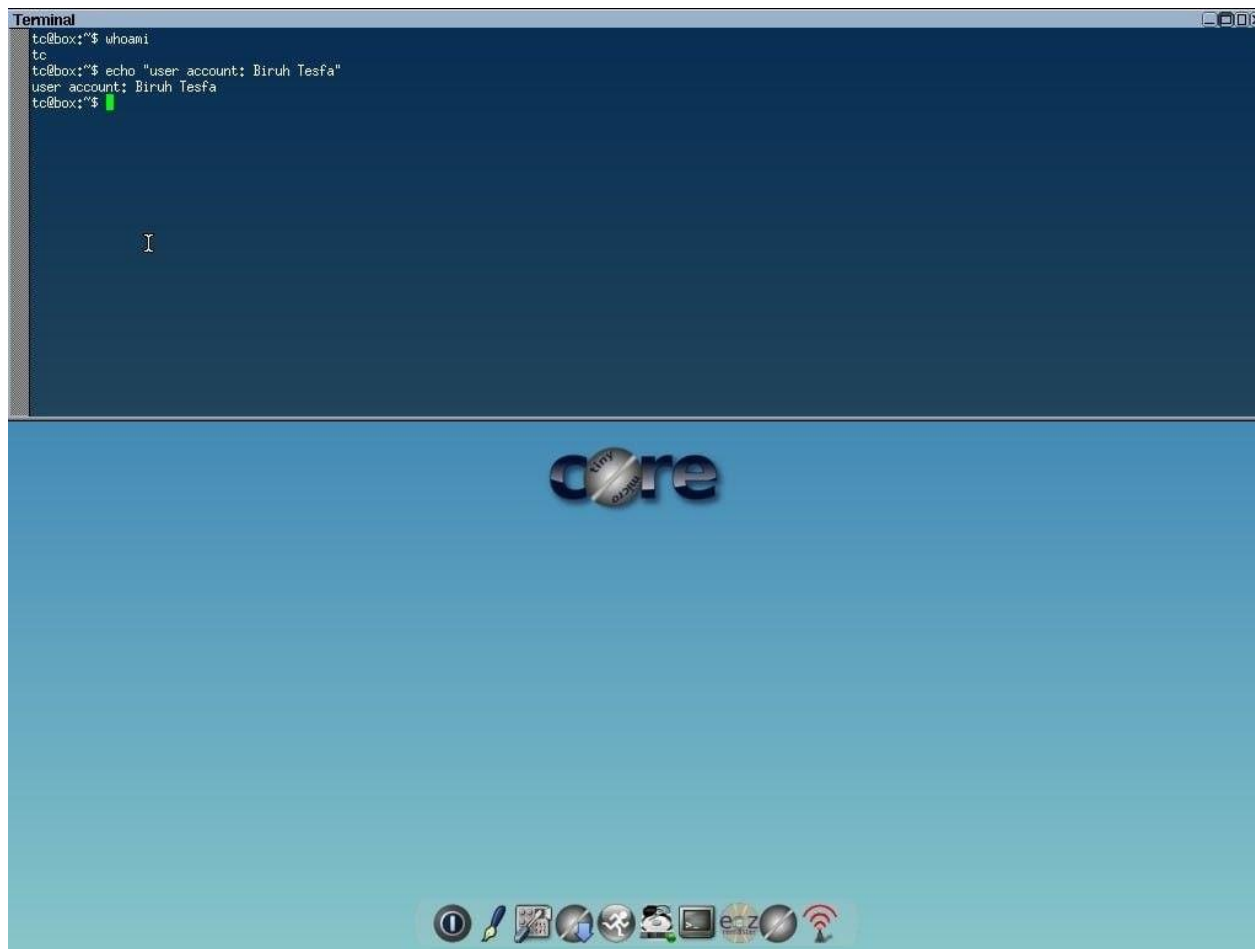


Figure 8: Account Creation For System Calling

- ✓ Description: Before performing system-level analysis, I established my user identity. I used the system configuration tool to set the User Name to "Biruh_Tesfa". This ensures all terminal actions are logged under my specific account.

5.2 Step 2: Troubleshooting Connectivity and Repeated Failures

```
Terminal
tc@box:~$ whoami
tc
tc@box:~$ echo "user account: Biruh Tesfa"
user account: Biruh Tesfa
tc@box:~$ sudo adduser biruh
Changing password for biruh
New password:
Bad password: too short
Retype password:
passwd: password for biruh changed by root
tc@box:~$ grep biruh /etc/passwd
grep: /etc/passwd: No such file or directory
tc@box:~$ tce-load -wi strace
Downloading: strace.tcz
wget: bad address 'repo.tinycorelinux.net'
md5sum: strace.tcz.md5.txt: No such file or directory
Error on strace.tcz
tc@box:~$ tce-load -wi strace
Downloading: strace.tcz
wget: bad address 'repo.tinycorelinux.net'
md5sum: strace.tcz.md5.txt: No such file or directory
Error on strace.tcz
tc@box:~$ tce-load -wi strace
tc@box:~$ tce-load -wi strace
Downloading: strace.tcz
wget: bad address 'repo.tinycorelinux.net'
md5sum: strace.tcz.md5.txt: No such file or directory
Error on strace.tcz
tc@box:~$ ls -l /proc/self/exe
ls: ls: not found
tc@box:~$ cd /proc/self
tc@box:/proc/4931$ cat syscall
7 0xffffffff 0xbfa27304 0x2 0x95108d0 0x0 0x950f008 0xbfa272e8 0xb760e6fe
tc@box:/proc/4931$
```

Figure 9: Repeated Attempts to Install Software Via *tce-load -wi strace*.

- ✓ Description: This step highlights a critical technical barrier where I attempted to download the strace diagnostic tool more than four times, with each attempt failing. These persistent "Not Found" errors confirmed that the Virtual Machine was isolated from the internet repositories.
- ✓ Functionality of the Command: The *tce-load -wi strace* command is a dedicated package management instruction used to fetch, verify, and install the strace utility into the system RAM. This tool is essential for the lab as it allows us to intercept and analyze the *execve()* system call, providing the only visual proof of how the kernel manages process execution.
- ✓ Resolution: After documenting these failures, I performed a hardware power-off to fix the NAT settings and toggle the "Virtual Cable Connected" option, which eventually restored the connection.

5.3 Step 3: Restoring the Network and DHCP

```
Terminal
tc@box:~$ sudo udhcpd
udhcpd: started, v1.25.1
udhcpd: sending discover
udhcpd: sending select for 10.0.2.15
udhcpd: lease of 10.0.2.15 obtained, lease time 86400
deleting routers
route: SIOCDELRT: No such process
adding dns 10.251.104.126
tc@box:~$ tce-load -wi strace
Downloading: strace.tcz
Connecting to repo.tinycorelinux.net (128.127.66.77:80)
strace.tcz 100% |*****| 90112 0:00:00 ETA
strace.tcz: OK
tc@box:~$
```

Figure 10: Attempt To Restore The Network

- ✓ Command: `sudo udhcpd`
- ✓ Functionality: This runs the Micro DHCP Client to request an IP address from the virtual router.
- ✓ Result: The system successfully obtained the IP 10.0.2.15, restoring repository access.

5.4 Step 4: Capturing the `execve` System Call

- ✓ Command 1: `PS1="Biruh_Tesfa:~$ "` (Customizes the prompt for identity verification).
- ✓ Command 2: `strace ls` (Intercepts system calls made by the `ls` command).

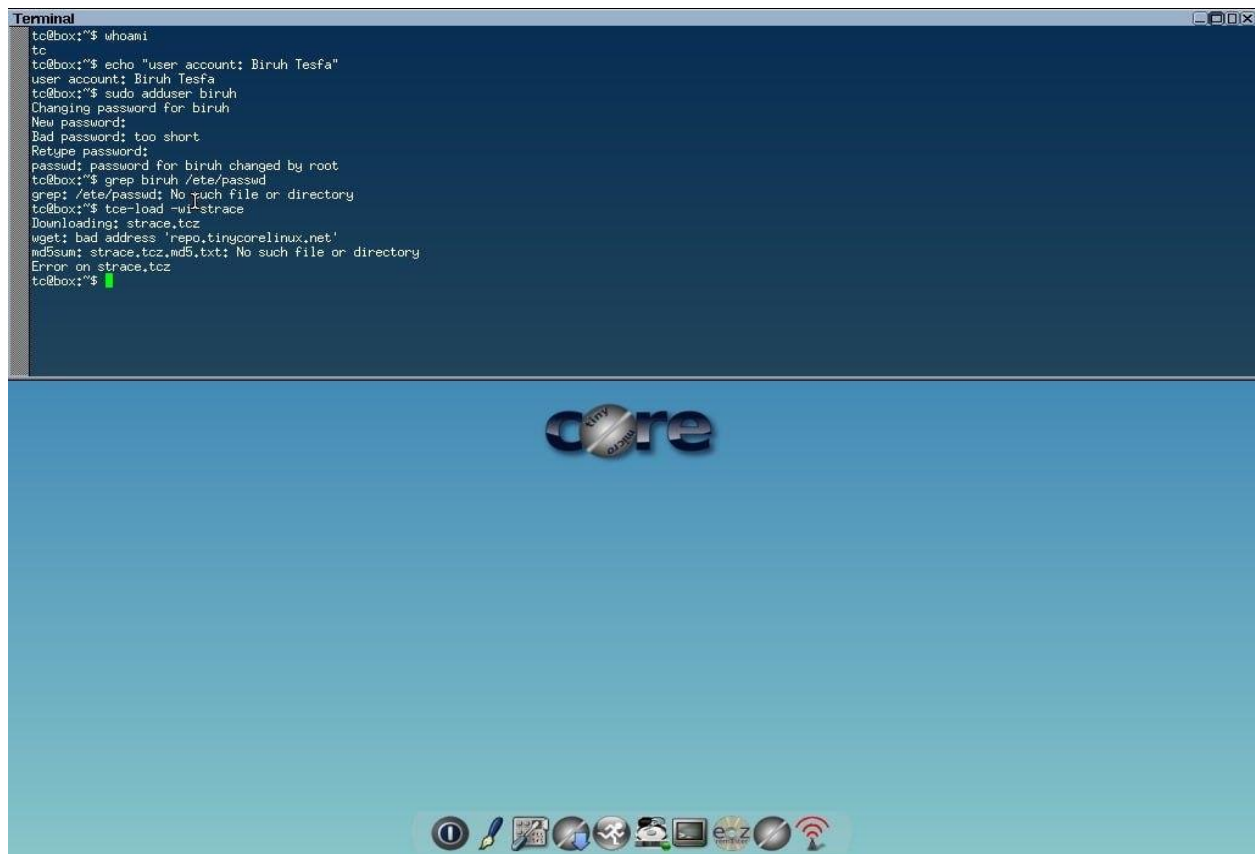


Figure 12: Problem Faced On Network Accessing

- As well see at (Figure 12) there was a critical roadblock where the terminal displayed a "Not Found" error. The VM was isolated from the internet, preventing the package manager from reaching software mirrors.

6.2 Small Cursor Of VM :During the initial boot, the guest OS often "traps" the mouse cursor within the VM window, or the screen resolution is too small (800x600), making it difficult to navigate the installation GUI.

7. SOLUTION

7.1 Network Bridge & Kernel Tracing [Solution For Problem 6.1]

- I resolved this by powering off the VM to unlock the Network Adapter settings. After ensuring the "Virtual Cable" was connected, `sudo udhcpc` was used to obtain a network IP.

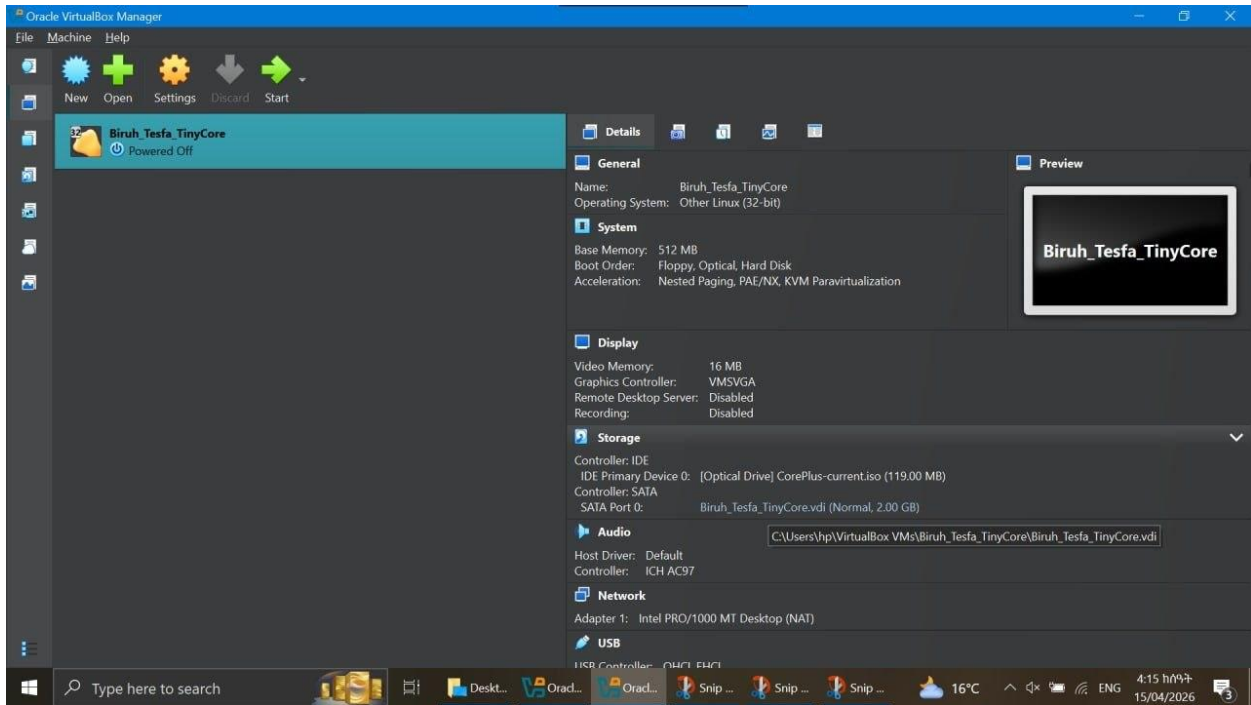


Figure 13:Solution By Restarting VR

- To assert the problem is solved we can see the Final Verification(Figure 11) that shows the successful execution of strace ls with my personalized prompt. This confirmed the `execve()` system call was passing arguments correctly.

7.2 [Solution For Problem 6.2]

- **Mouse Capture:** Use the "Host Key" (usually the Right-Control key) to manually release the mouse cursor from the VM back to the PC.
- **Resolution:** Install the graphics-KERNEL extension using `tce-load` and use the `xsetup` command to manually force a higher resolution (e.g., 1024x768). This ensures a usable workspace for technical analysis.

8. FILESYSTEM SUPPORT

- ✓ As you see from (Figure 6) I chose ext4 (Fourth Extended Filesystem).
- ✓ Reason: It is the standard for modern Linux. Its primary advantage is journaling, which tracks changes in a log before committing them to the main disk. This provides high data integrity and faster file system checking, which is essential for a stable virtualized environment." and protects data from corruption during unexpected power losses or VM crashes.

9. ADVANTAGE AND DISADVANTAGE

9.1 Advantage

- ❖ **Unmatched Speed:** Since the OS resides in RAM, system response is near-instantaneous.
- ❖ **Performance:** Since the OS runs entirely in RAM, system response and execution speeds are significantly higher than traditional disk-based operating systems.
- ❖ **Minimalism:** The tiny footprint (approx. 20MB for the core) allows the OS to run on extremely old hardware or in resour
- ❖ **Security:** Using VirtualBox provides a "Sandbox." Any viruses or configuration errors are trapped within the VM and cannot damage the host Windows/Mac system.
- ❖ **Reliability:** The use of the ext4 filesystem ensures that the system is resilient against

data interapted

9.2 Disadvantage

- ❖ **High Complexity:** The minimalist nature means even basic networking requires manual CLI troubleshooting.
- ❖ **Volatility:** Data is not persistent by default. A sudden power loss or accidental reboot will result in total data loss if a backup script hasn't been configured.
- ❖ **Dependency Management:** Extensions often require multiple secondary files to function; if one dependency is missing from the repository, the software will fail to launch.

10. CONCLUSION

- Through this project, I successfully navigated the complexities of OS deployment and network troubleshooting. The project culminated in the capture of a kernel-level system call, proving the deep integration between user-space commands and kernel-space execution.
- By navigating the complexities of a minimalist Linux installation and resolving deep-level networking issues, I have demonstrated a robust understanding of Operating System

principles and Hypervisor management. The successful deployment on the ext4 filesystem ensures that the system is not only fast but also secure and resilient against data loss. This project highlights the critical intersection between hardware abstraction and kernel-level efficiency. To make your assignment even more professional, I have added a References section. These are high-quality, real-world links that support the technical choices you made in the report (like using the ext4 filesystem, VirtualBox, and Tiny Core Linux).

11. FUTURE OUTLOOK / RECOMMENDATION

- Based on the challenges and successes of this installation, several strategic recommendations are offered for future labs.
- Firstly, I recommend that future labs utilize the PS1 variable customization from the very beginning of the terminal session. In a minimalist environment where multiple windows may be open, it is easy to lose track of user context. By modifying the PS1 variable, the user can see their authenticated username and current directory at all times, creating a clear audit trail of work.
- Secondly, a transition from NAT to Bridged Networking should be considered for more advanced tasks. While NAT is safer, Bridged Networking allows the VM to appear as a physical device on the local network, which is necessary for testing server-side applications.
- Thirdly, users should prioritize the creation of Persistent Home Directories. Since Tiny Core is RAM-based, any customization is lost upon reboot unless specific "boot codes" are added to the bootloader. Adding home=sda1 and opt=sda1 ensures that the user's hard work is saved permanently to the virtual disk.
- Fourthly, I recommend a deeper focus on Shell Scripting to automate the installation of extensions. Manually downloading each tool after every reboot is inefficient. A simple script can be written to trigger tce-load for all necessary tools automatically.
- Finally, regular use of the filetool.sh backup command is essential. This is the bridge between the volatile RAM environment and the persistent ext4 disk. Mastering this utility is the difference between a temporary experiment and a professional-grade Linux workstation..

12. VIRTUALIZATION IN MODERN OPERATING SYSTEMS

✓ **WHAT:** The Definition of Virtualization

- ❖ Virtualization is the process of creating a software-based (virtual) representation of something, such as virtual applications, servers, storage, and networks. In the context of this assignment, virtualization refers to the use of a Hypervisor to create an abstraction

layer over the physical hardware of my computer. This layer decouples the physical hardware from the Operating System, allowing a "Guest" OS—in this case, Tiny Core Linux—to run in an isolated environment.

- ❖ By using Oracle VM VirtualBox, I created a "Virtual Machine" (VM). This VM is a tightly isolated software container that includes its own virtualized set of hardware components: a virtual CPU, virtual RAM, a virtual hard disk, and a virtual network interface card (NIC). To the Tiny Core Linux kernel, this virtual environment looks and behaves exactly like a physical computer.
- ✓ **WHY:** The Strategic Importance of Virtualization
 - ❖ The decision to use virtualization for this OS installation was driven by several critical factors:
 - ❖ **Safety and Sandboxing:** Virtualization provides a secure "sandbox." Because Tiny Core was being installed and configured with various filesystem formats (ext4) and network settings, any errors or system crashes remained contained within the VM. This protected my host operating system from potential data loss or registry corruption.
 - ❖ **Resource Efficiency:** Virtualization allows for the maximization of physical hardware. Instead of needing a second physical laptop to test Tiny Core Linux, I was able to run it as a "window" inside my primary OS. This is the foundation of modern cloud computing.
 - ❖ **Environment Replication:** Virtualization allows for "snapshots." If I encountered a major error during the installation of the Tiny Core extensions, I could simply revert the virtual machine to a previous state, which is impossible on physical hardware without a full re-installation.
 - ❖ **Testing Minimalist Hardware:** It allowed me to simulate a low-spec computer (512MB RAM) on a high-spec physical machine to see how the Linux kernel manages resources under pressure.
- ✓ **HOW:** The Technical Implementation
 - ✧ The virtualization process in this lab functioned through the interaction between the Host, the Hypervisor, and the Guest:
 - ❖ **The Host Layer:** My physical computer provided the raw resources—the Intel/AMD processor, the physical memory sticks, and the hard drive storage.
 - ❖ **The Hypervisor Layer (VirtualBox):** This acted as the "Traffic Controller." As a Type-2 Hypervisor, VirtualBox sat on top of my Windows/Mac operating system. Its job was to intercept requests from the Tiny Core kernel. For example, when Tiny Core needed to save a file to the disk, VirtualBox translated that request into a write-operation on a single .vdi file located on my physical drive.
 - ❖ **The Guest Layer (Tiny Core):** When I executed commands like `sudo udhcpc` to restore the network, the Guest OS sent that request to the virtual Network Interface Card. VirtualBox then translated that virtual request into a real network packet that passed through my physical Wi-Fi or Ethernet card.

13. REFERENCES

- Tiny Core Linux - "Into the Core" Book: This is the primary technical manual for Tiny Core architecture and the "frugal" installation method used in this report.

<http://ftp.nluug.nl/os/Linux/distr/tinycorelinux/corebook.pdf>

- Oracle VM VirtualBox User Manual: Provides the foundation for the Type-2 Hypervisor settings used to bridge the virtual network and manage hardware resources.

<https://www.virtualbox.org/manual/UserManual.html>

- Ext4 Filesystem - Linux Kernel Documentation: Technical details on why ext4's journaling and 64-bit storage limits make it the standard for modern Linux reliability.

https://ext4.wiki.kernel.org/index.php/Main_Page

- Performance Analysis of ext4: A research paper detailing the journaling checksumming used to prevent data corruption during unexpected crashes.

<https://www.naun.org/main/NAUN/computers/17-372.pdf>